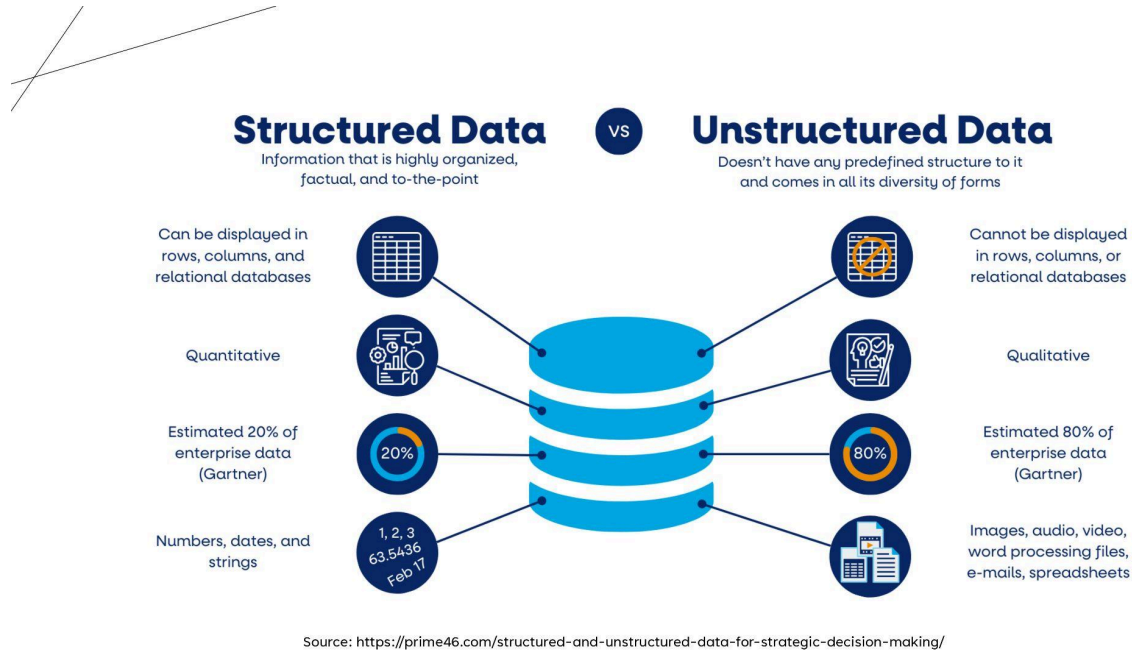




STRUCTURED DATA

Structured data is like a well-arranged spreadsheet or table.

- Everything is labeled and organized
- You have rows and columns
- Each column has a clear meaning

Example:

Name	Age	Marital Status	Salary
Rauf	24	Unmarried	0

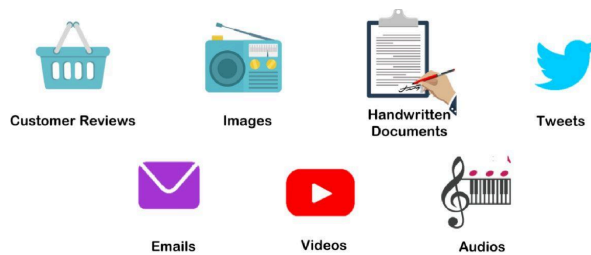
- Easy to search, filter, and analyze
- Stored in systems like relational databases (RDBMS)
- You already know what each value means (e.g., salary = 0)

UNSTRUCTURED DATA

Unstructured data is like a box full of random stuff.

- No fixed format (no rows/columns)
- Not neatly labeled
- Harder for computers to understand directly

Examples:



Even though it's "messy," it still contains a lot of useful information.

Why Are Images & Videos Unstructured?

Images and videos don't fit into tables. Instead, they are made of numbers (pixels).

Examples

- An image is just a grid of numbers:
A 224×224 RGB image = $224 \times 224 \times 3 = 150,528$ values
But here's the catch:
These numbers don't directly mean anything to us
(125, 200, 30) $\neq$ "cat" or "dog"
The meaning is hidden, not explicit.
- A video is just many images played quickly:
1 minute video at 30 FPS = 1800 images
Now imagine the data size and complexity

The Real Challenge

1. No Direct Meaning

Structured data:

Salary = 0 → clear meaning

Image data:

Pixel values → no direct meaning

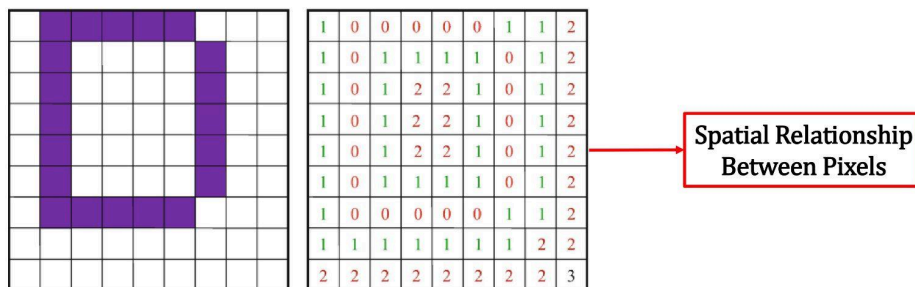
So the model must learn what patterns mean

2. Spatial Relationships (Images)

Pixels are not independent!

- Nearby pixels are related
- They form edges, shapes, textures

A single pixel means nothing alone its neighbors matter



3. Temporal Relationships (Videos)

Videos add time

Frame 1 → Frame 2 → Frame 3

Motion and context evolve over time

Understanding a video = understanding change over time

Why We Need Special Models (like CNNs)

Because of all this complexity:

- Huge number of values
- Hidden meaning
- Spatial + temporal relationships

We cannot treat images like tables.

We need models that:

- Automatically extract features
- Understand patterns in space (images)
- Understand patterns in time (videos)

That's exactly why models like CNNs (Convolutional Neural Networks) are used.

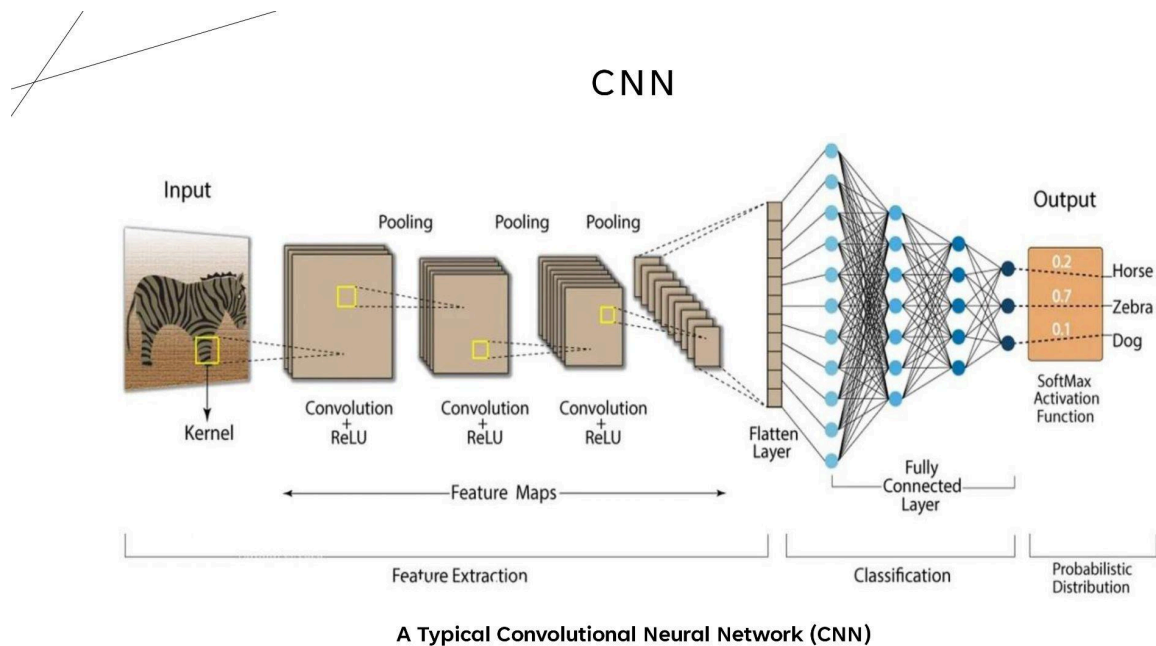
CNN

CNN is a deep learning model specifically designed to process grid-like data like images and videos. It automatically extracts spatial features, which means it learns to find the important parts of a picture (like eyes or wheels) all on its own.

The Three Secret Ingredients of CNNs

- **Local Connectivity:** Instead of looking at every pixel at once, the computer looks at a small area or neighborhood of pixels at a time.
- **Weight Sharing:** Instead of every pixel having its own unique connection, a CNN uses a small kernel (like a 3 x 3) that slides across the whole image. Because the same filter is used across the whole image, the number of parameters is greatly reduced, which lowers the computational cost, making it faster and more efficient.

- **Hierarchical Feature Learning:** This is like building a house.
 - Early Layers: Detect simple things like straight lines or edges.
 - Mid Layers: Combine those lines to see shapes (like circles or squares).
 - Deep Layers: Put the shapes together to see complex things (like a zebra's stripes or a face).



11

Let's look at the image

Step 1: Feature Extraction

This is where the computer tries to understand what's in the picture.

- **The Kernel:** See that little yellow square on the zebra's leg? That's a Kernel (or filter). It slides across the image looking for specific patterns.
- **Convolution + ReLU:** This is a math step where the computer highlights the patterns it found. ReLU is like a "filter" that says, "If this pixel isn't important, just turn it to zero (black)."

- **Pooling:** This shrinks the image to keep only the most important information, making it easier for the computer to handle.

Step 2: Classification

Once the computer has found all the patterns, it needs to guess what it's looking at.

- **Flattening:** The computer takes all those 2D patterns and lines them up in a single long row (like a list of clues).
- **Fully Connected Layer:** This layer looks at the final list of clues and says, "Okay, I see stripes, four legs, and a tail... it's probably a zebra!"
- **SoftMax (The Final Vote):** This turns the computer's guess into a probability. In the diagram, it's 70% (0.7) sure it's a zebra!

WHY NOT MLP?

Two main reasons MLPs fail at computer vision:

- **Explosive Number of Parameters:** Imagine a small color photo. As we learned, it has over 150,000 pixels. In an MLP, every single pixel must connect to every single "neuron" in the next layer. This creates millions of connections (parameters).
 - Analogy: It's like trying to memorize the exact location of every single grain of sand on a beach just to recognize that it's a beach. It's too much work for the brain!
- **Lack of Spatial Reasoning:** MLPs force an image to "flatten" into a 1D line of numbers.
 - The Problem: When you turn a 2D picture into a 1D line, the pixels that were "neighbors" (like the two eyes of a dog) are now far apart. The computer loses the "shape" of the object.

WHY CNN COMPUTATIONAL EFFICIENCY?

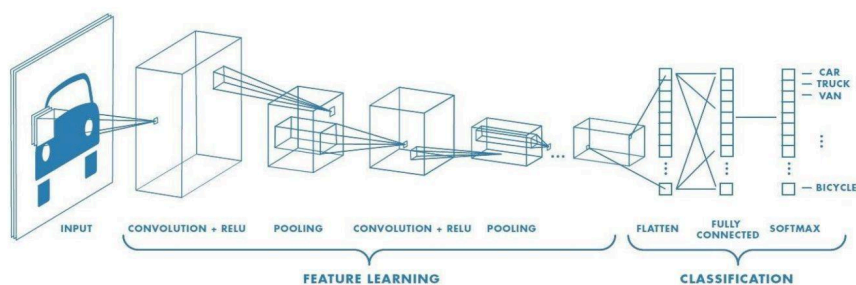
Fully Connected Neural Networks Struggle with High-Dimensional Data. For example, A 224×224 RGB image has 150,528 features. As a result, a fully connected layer would require millions of parameters (150529 for each neuron in the first hidden layer)

This leads to:

- Overfitting due to excessive trainable weights.
- High computational cost and memory requirements.

Computational Efficiency in CNNs: Convolutional filters are shared across the image, significantly reducing the number of trainable parameters. A 3×3 kernel has only 9 parameters (27 parameters per filter in RGB), compared to thousands in a fully connected network. So, even if 100 filters are used, the parameter count will be just 900 (+100 for the bias term) for the first hidden layer. Compare that to an MLP with 100 neurons in the first hidden layer for the simple 28×28 MNIST dataset images. It would have $784 \times 100 = 78,400$ parameters.

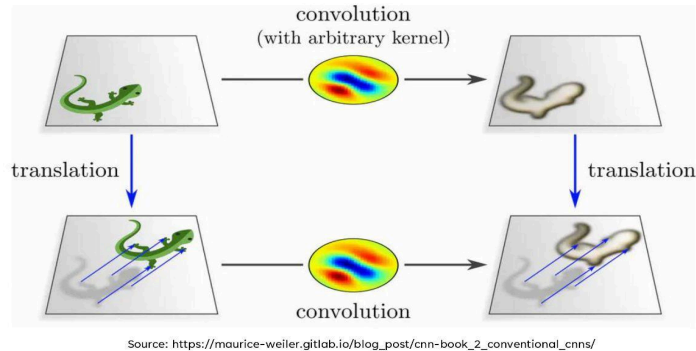
SPATIAL AWARENESS AND RECEPTIVE FIELDS



- **Receptive Field:** This is the small "patch" of the image that a single neuron is looking at.

- How it works: Early layers look at tiny patches (to see edges). As we go deeper into the network, the "field" gets larger, allowing the network to see whole shapes like a car or a bicycle.

Translational Invariance



15

In the image, we see a lizard moving around a frame.

- Translational Invariance: This is a fancy way of saying: "A lizard is a lizard, whether it's in the top-left corner or the bottom-right corner of the photo."
- Why CNNs win: Because the filter slides (convolves) across the entire image, it will "catch" the feature (like a lizard's tail) no matter where it is located. An MLP would have to learn what a tail looks like in every single possible position separately!

THE CONVOLUTION PROCESS

Convolution is the process of sliding a filter (kernel) over an image and computing dot products to produce a feature map. This is basically a new version of the image where the important parts (like edges or curves) are glowing brightly, and the unimportant parts are dimmed.

Convolution Operation Equation (2D Cross-Correlation Form)

$$(I * K)_{i,j} = \sum_{m=1}^m \sum_{n=1}^n I_{i+m,j+n} \cdot K_{m,n}$$

In the equation, what each part represents is,

I(input): This is your original image, the big grid of pixels.

K (kernel): This is your small filter (e.g., a 3 x 3 square).

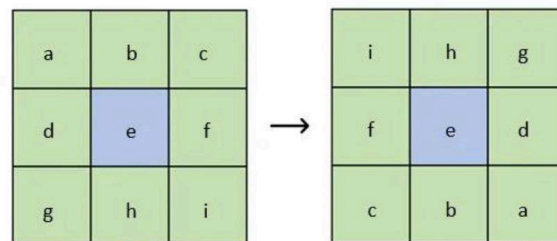
(i, j): This is the specific "address" (row and column) of the pixel we are currently looking at.

(m, n): These are just counters that help the computer move through every tiny cell inside the filter.

The Intuition: The computer places the filter over a patch of the image, multiplies the overlapping numbers, and adds them into one single total. If the total is high, the computer has "found" the pattern the filter was looking for!

KERNELS

A **Kernel** is a small matrix used to apply effects like blurring, sharpening, edge detection, etc., on an image. It operates by sliding over the image, performing element-wise multiplication with the image pixels, and summing the results to produce a new pixel value. Kernels are also used in CNNs.



Kernel

Flipped Kernel

Source: https://medium.com/@timothy_terati/image-convolution-filtering-a54dce7c786b

Think of the kernel as the eyes or the expert lens of the neural network. Without it, the computer would just be looking at a giant, meaningless wall of numbers. Kernel is the most important tool in CNN because,

1. It acts as a Pattern Finder

The computer doesn't naturally know what a line, a curve, or a circle is. A kernel is a small matrix (like a 3 x 3 grid) designed to react when it sees a specific pattern.

- One kernel might be an expert at finding vertical lines.
- Another might be an expert at finding horizontal edges.
- As these kernels slide over the image, they light up (produce high numbers) whenever they find their specific pattern.

2. It Makes the Computer Efficient

Instead of having a unique expert for every single pixel in an image which would require millions of connections we use the same kernel to scan the entire image.

- The Analogy: If you are looking for a specific word in a book, you don't need a different pair of eyes for every page. You use the same eyes to scan page after page.
- Because the kernel is small and its values are shared across the whole image, it drastically reduces the "explosive number of parameters" that would otherwise crash the computer.

3. It Learns Spatial Features Automatically

In the old days of computer science, humans had to manually program filters to find edges. With CNNs, the kernels learn what they need to look for during training.

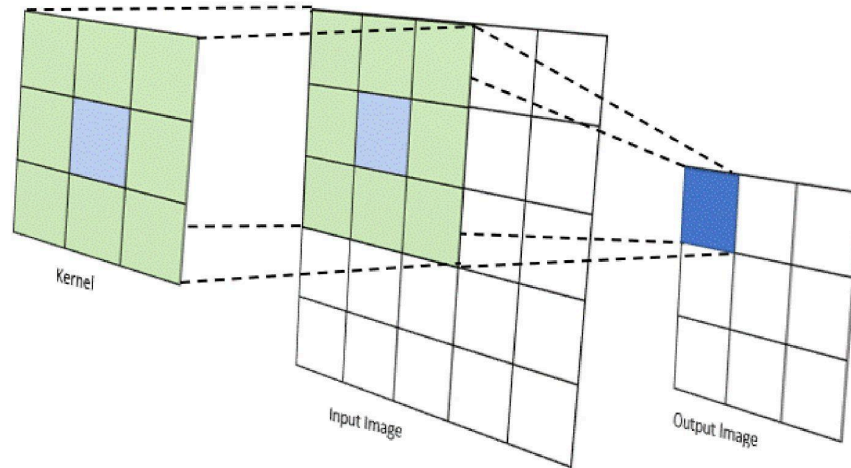
Early Layers: Kernels learn simple things like textures.

Deep Layers: Kernels learn to recognize complex objects like faces or animals by combining the simple patterns found by earlier kernels.

4. It Provides Translational Invariance

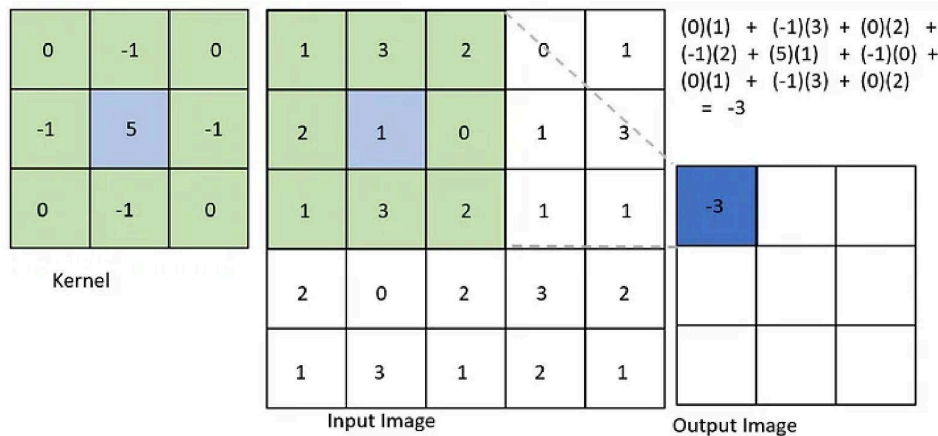
Since the kernel slides (convolves) across every location, it doesn't matter where an object is in the picture. If a kernel is looking for a pattern like a bird's eye, it can detect that pattern whether the bird is in the top-left corner or the bottom-right.

A filter (kernel) is systematically slid over a whole image.



Source: https://medium.com/@timothy_terati/image-convolution-filtering-a54dce7c786b

Pointwise multiplication takes place on each location, and the sum of them is assigned to the corresponding center pixel of the output image (feature map)..



Source: https://medium.com/@timothy_terati/image-convolution-filtering-a54dce7c786b

The filter may not always be given explicitly. For example, a question might simply ask to use a 3×3 mean or median filter. In that case, we take a 3×3 region from the image and calculate its mean or median value.

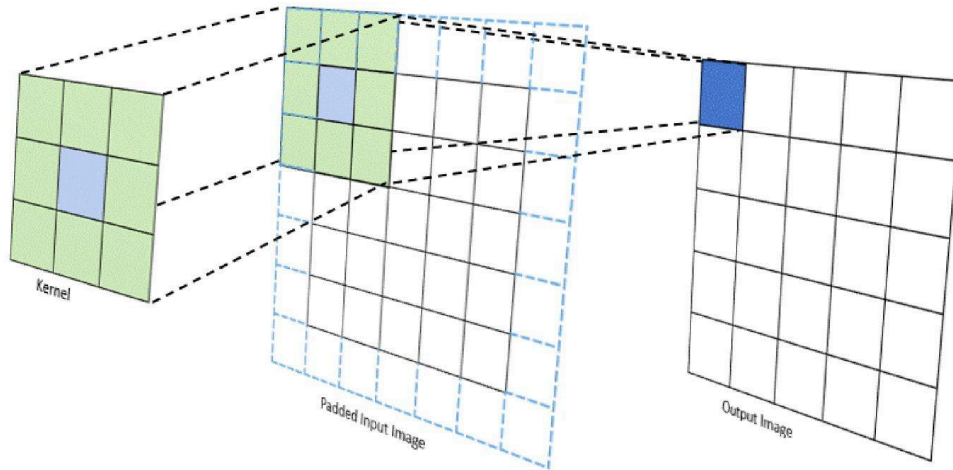
For example, in this image, if we are asked to use a mean or median filter, then the value at position (0,0) in the output image would be 2.

Mean = $(1+3+2+2+1+0+1+3+2)/9 = \text{round}(1.667) = 2$

Median = 0 1 1 1 **2** 2 2 3 3 = 2

THE CONVOLUTION PROCESS (PADDING)

As we can see, Convolution may reduce the size of the image/signal. **Padding** extra layers of pixels around the borders can be done to preserve the original image's dimensions or to get any arbitrary shape. **Zero Padding, Edge Padding..** These are some of the popular padding methods.



Source: https://medium.com/@timothy_terati/image-convolution-filtering-a54dce7c786b

22

Think of Padding as a "protective border" we add around our image before we start the convolution process.

When we slide a filter over an image, two things naturally happen that we might not want:

- The image shrinks: Every time a filter (kernel) passes over an image, the output is slightly smaller than the input.
- Edge information is lost: Pixels in the corners and edges are only touched by the filter once, while pixels in the middle are touched many times. This means the computer "ignores" the edges of your photo.

Padding solves this by adding extra "dummy" pixels around the outside of the image.

Zero Padding

The most common type is Zero Padding. We surround the image with a border of zeros (which represents black/empty space).

- **Maintaining Size:** By adding this border, the filter can "start" outside the actual image, ensuring the resulting feature map is the exact same size as the original image.
- **Equal Treatment:** Now, the pixels that were on the very edge of the original image get scanned multiple times, just like the pixels in the center.

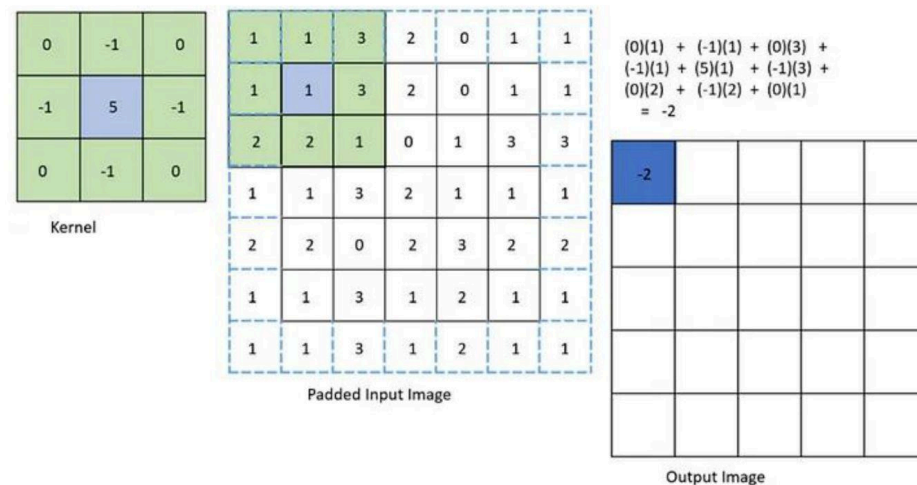
Why is Padding Necessary?

In deep networks, we use many layers. If we didn't use padding:

- **Shrinkage:** A 32x32 image might become 30x30, then 28x28, then 26x26. Eventually, the image would disappear entirely!
- **Border Neglect:** The network would never learn the features located at the very corners of your images (like the tips of a cat's ears).

Mainly we use padding when edges of the image contain useful information.

Padding 1 layer of pixels on each side preserves the dimension of the original image in the case of a 3x3 kernel.

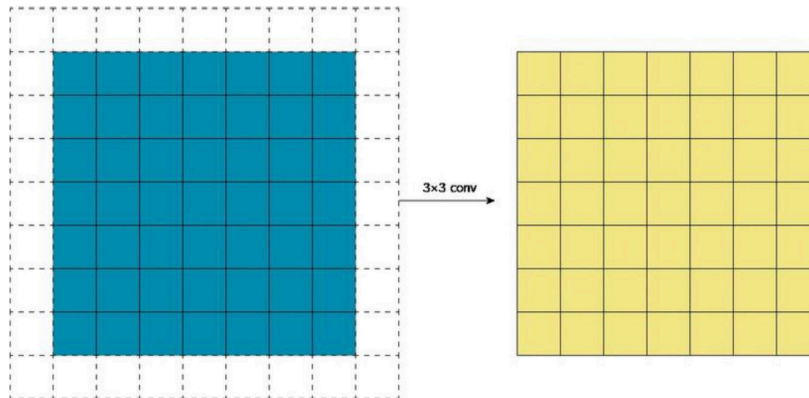


Source: https://medium.com/@timothy_terati/image-convolution-filtering-a54dce7c786b

THE CONVOLUTION PROCESS (STRIDES)

Stride defines how many pixels the filter (kernel) moves across the input image at each step. It controls how much we slide the filter horizontally and vertically during convolution. The animation below shows the convolution process with a **Stride = 2**.

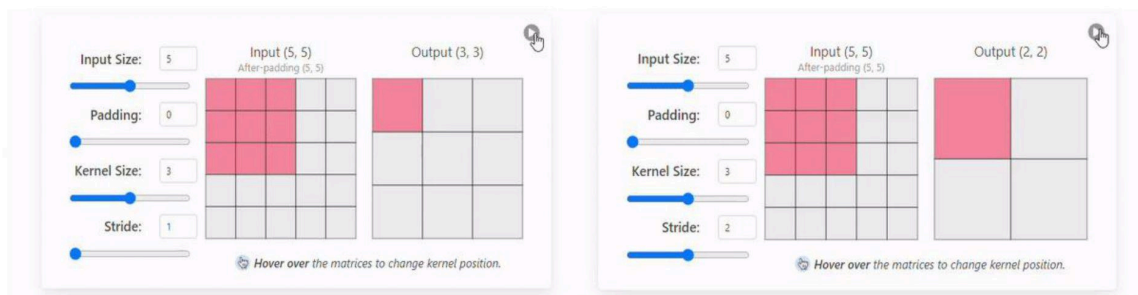
Clearly, increasing the number of strides results in smaller images/feature maps. This can be used to reduce spatial dimensions and thus, computation cost.



Source: <https://towardsdatascience.com/intuitively-understanding-convolutions-for-deep-learning-1f6f42faee1>

THE CONVOLUTION PROCESS (STRIDES)

Observe the animations below to understand how different values of strides affect the dimensions of the output feature map.



Stride = 1

Stride = 2

Source: <https://poloclub.github.io/cnn-explainer/>

Why do we change the Stride to skip pixels and lose information?

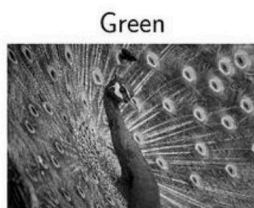
There are two main reasons:

- **Reducing Image Size (Downsampling):** If we have a massive high-definition image, processing every single pixel is slow and takes up a lot of memory. By using a larger stride, we shrink the image quickly, keeping only the most important features.

- **Overlapping Information:** With a stride of 1, the "receptive fields" (the patches the kernel looks at) overlap a lot. If we want the network to look at more distinct, non-overlapping parts of the image to get a "broader" view, we increase the stride.

CHANNELS

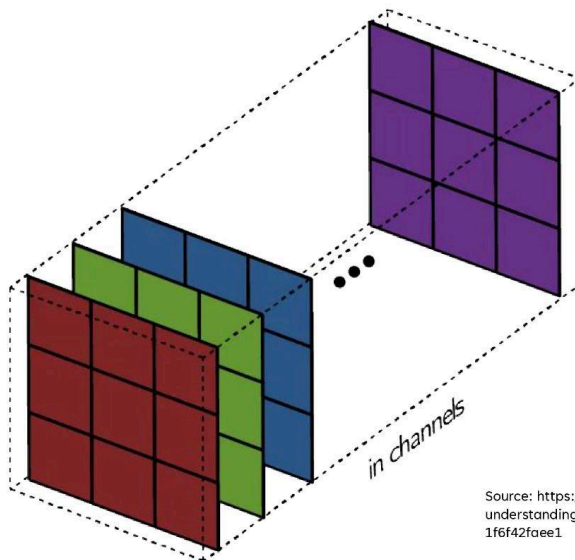
A **Channel** is like a separate layer of information in the image. A grayscale image has **1** channel (only **intensity**). An RGB image has **3** channels: **R (Red)**, **G (Green)**, and **B (Blue)**. Some images (e.g., medical, hyperspectral) can have dozens of channels. We can think of channels like stacked sheets of paper: each sheet holds different information about the same position. For example, in RGB images, each channel contains information on different wavelengths of light (different colors).



Source: <https://towardsdatascience.com/intuitively-understanding-convolutions-for-deep-learning-1f6f42faee1>

CHANNELS

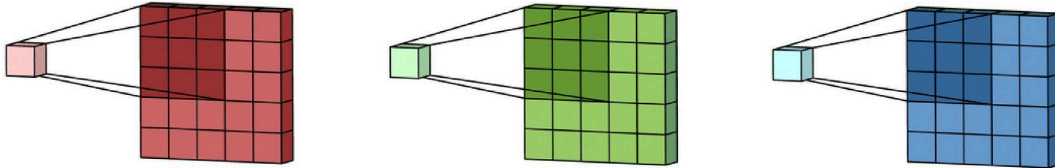
Instead of width $\times$ height (e.g., 224×224), an RGB image structure is **width $\times$ height $\times$ depth [channels]** ($224 \times 224 \times 3$). This is true for any image with multiple channels.



Source: <https://towardsdatascience.com/intuitively-understanding-convolutions-for-deep-learning-1f6f42faee1>

CONVOLUTION ACROSS CHANNELS: KERNELS vs FILTERS

A single filter must process all channels together, not separately. **Each 3D filter will now have the same number of 2D kernels as the number of channels in the input image (One kernel for each channel).** So, an RGB image will need filters with 3 kernels (like channels in the image) each.



Source: <https://towardsdatascience.com/intuitively-understanding-convolutions-for-deep-learning-1f6f42faee1>

30

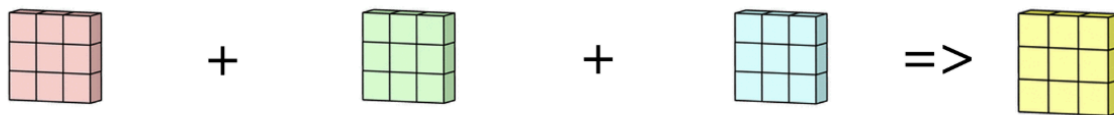
KERNEL VS FILTER

Kernel: A kernel is a small matrix (for example, 3×3) that slides over an image or data. It performs element-wise multiplication and summation operations to produce a new output.

Filter: A filter is generally made up of multiple kernels. For example, an RGB image has three channels, so a filter contains three kernels, one for each channel.

Basically, a filter is a collection of kernels. When a filter is applied in a convolutional layer, each kernel in the filter is convolved with its corresponding input channel. The results are then added together to create a single output image or feature map. For example, in a RGB image,

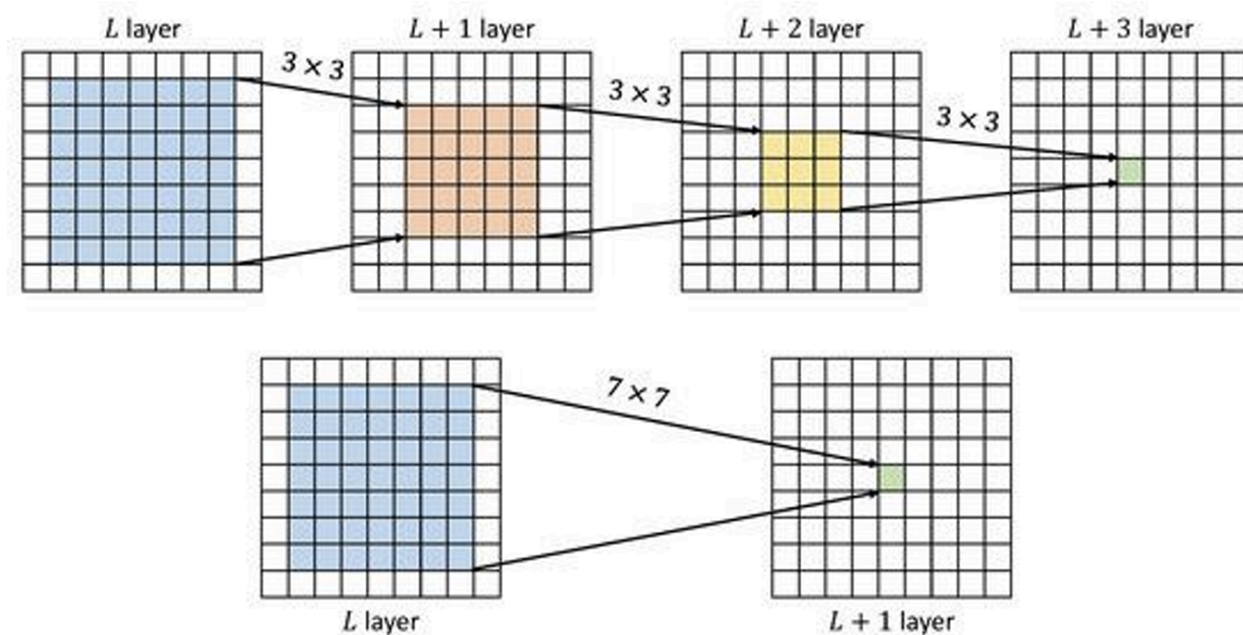
- The Red kernel scans the Red channel.
- The Green kernel scans the Green channel.
- The Blue kernel scans the Blue channel.



RECEPTIVE FIELD

The receptive field is the specific region of the original input image that affects a particular neuron's value.

- **Starting Small:** In the first layer, a neuron only "sees" a tiny patch, usually the same size as the filter (like a 3×3 area).
- **Growing Large:** As we go deeper into the layers or when the size of the filter is larger, the receptive field grows. This is because each neuron in a deeper layer is looking at several neurons from the previous layer, which in turn were looking at different parts of the original image.



Source: <https://doi.org/10.3390/app13063403>

We can see a great step-by-step visualization:

Layer L: Sees a small 3 x 3 patch.

Layer L+3: By the time we reach deeper layers, that single neuron is indirectly "seeing" a much wider context of the original input.

Filter Size Matters: You can also see that using a larger filter (like 7 x 7) makes the receptive field grow much faster than a 3 x 3 filter.

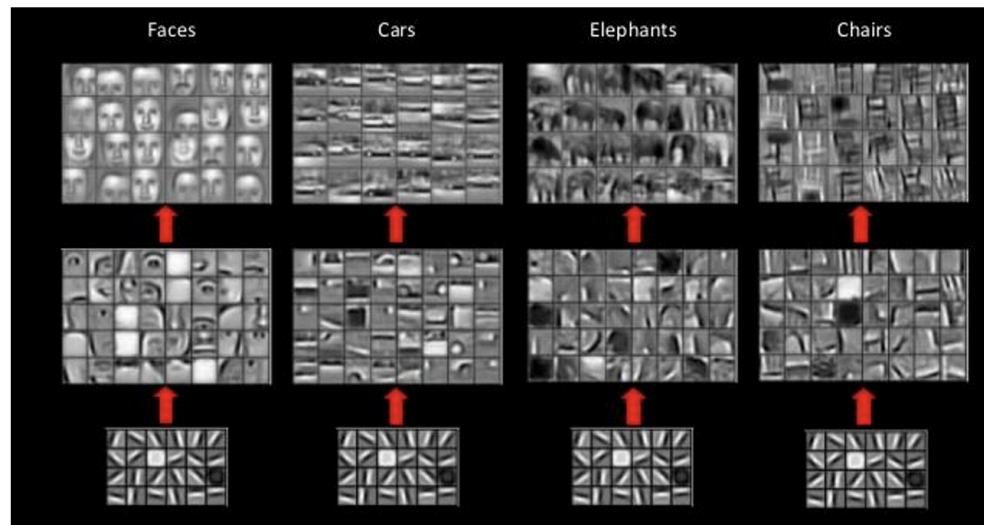
WHY RECEPTIVE FIELDS ARE IMPORTANT

The receptive field measures how much of the original input a neuron 'sees'. Deeper neurons have larger receptive fields, enabling detection of complex and large-scale patterns. This process can be compared to the foveal and peripheral vision of the human visual system.

- **Small Receptive Fields** act like Foveal Vision. When we read a word, our eyes focus on just a few letters. We're picking up fine details. This is like the early CNN layers with small receptive fields. These detect edges, textures, colors and very local features.
- **Large Receptive Fields** play the role of Peripheral Vision. As we look around a scene, our peripheral vision helps us understand the overall layout where the objects are, who's standing where. This is like deeper CNN layers. They integrate signals from many small details to build up semantic understanding. They detect faces, animals, traffic signs, etc.

In short, CNNs work like our visual system. We first see local details (edges, corners) using smaller receptive fields and gradually build a global picture (faces, objects) using larger receptive fields. Also, this process enables detecting hierarchical features as we go deeper into the CNN. From lines/edges (earlier layers) → shapes → objects (deeper layers), like from letters → words → sentences. That's why convolutional layers are known as feature extractors.

The image below shows how CNN layers capture lower-level features like lines/edges (local patterns) in the earlier layers and gradually move towards capturing more complex features like eyes, faces (global shapes/objects) as we go deeper into the network.

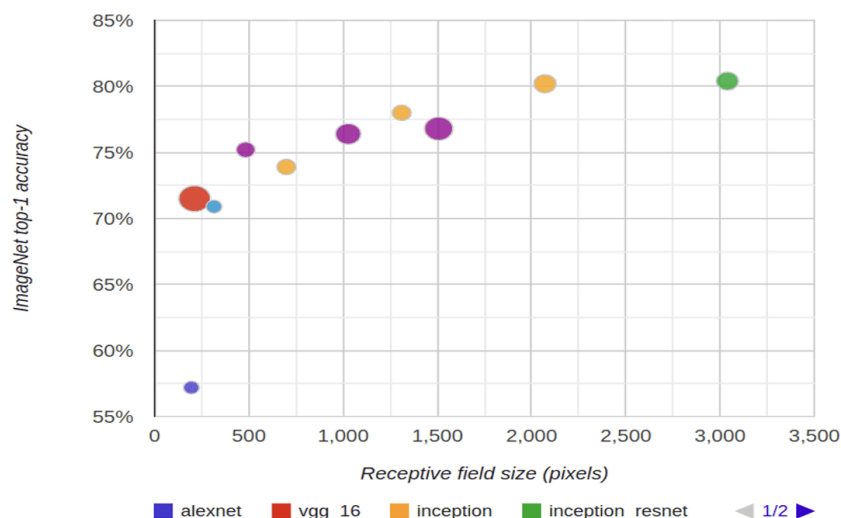


Source: <https://www.linkedin.com/pulse/deep-learning-understanding-receptive-field-computer-n-bhatt>

You can see this hierarchy in the image where simple lines at the bottom eventually combine into complex objects like chairs and elephants at the top.

High-Level Recognition

In the graphic below, we observe a logarithmic relationship between classification accuracy and receptive field size, which suggests that **large receptive fields are necessary for high-level recognition tasks, but with diminishing returns.**



Source: <https://theaisummer.com/receptive-field/>

38

A graph comparing accuracy to receptive field size. Larger receptive fields are generally necessary for high-level recognition tasks. But there are diminishing returns. Making the field infinitely large doesn't keep making the computer infinitely smarter; it just needs enough context to understand the object.

LAYERS OF CNN: THE BUILDING BLOCKS

An end-to-end **Convolutional Neural Network (CNN)** typically contains several types of layers, each with different mechanism and purpose. Each layer transforms the input and feeds richer representations forward. Together, these layers form the powerful CNN architecture which can perform various ML tasks.

Common Layers of a CNN

- Convolutional Layer
- Pooling Layer
- Flattening Layer
- Fully Connected (Dense) Layer
- Activation Layer
- Batch Normalization Layer
- Dropout Layer

Some of these layers (like activation or dropout layers) are sometimes not explicitly mentioned and considered part of other layers. But roughly, these are the most common layers used in a CNN.

40

CONVOLUTIONAL LAYER

Convolutional layers are the fundamental building blocks of CNNs. These layers perform a critical mathematical operation known as convolution (As we learned already). This process entails the application of specialized filters known as kernels, that traverse through the input image to learn complex visual patterns.

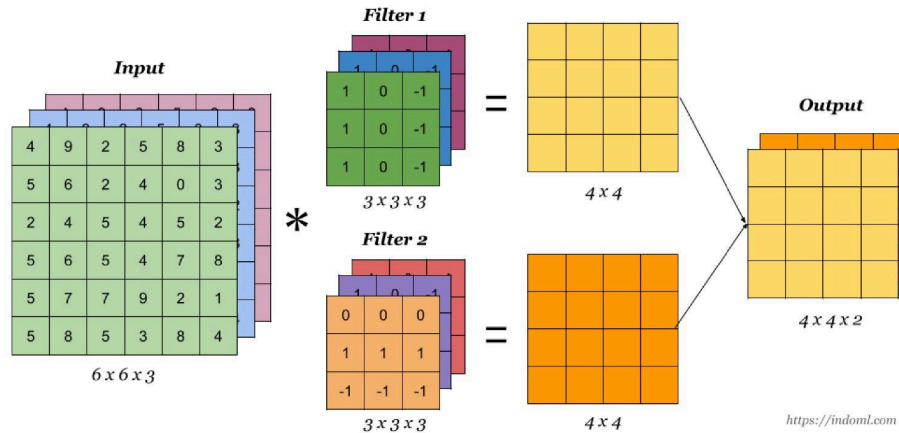
Purpose: Applies not one but **a set of filters** (learned kernels) over the input, producing multiple feature maps as output. These feature maps help to:

- **Detect local patterns** within the receptive field (e.g., edges, textures, shapes).
- **Learns what to look for** (e.g., vertical edge, circle, eye, etc.).



CONVOLUTIONAL LAYER

The image below underlines how multiple filters help to extract different features. By applying **multiple filters**, a convolutional layer creates **a set of feature maps**, each representing a different aspect of the input data.



While convolution layers can shrink an image, their real job is Feature Extraction (finding those clues). Pooling, on the other hand, has one main mission: Dimensionality Reduction. It wants to make the data smaller and more manageable without losing the "essence" of what the detective found.

What is Pooling?

Imagine you have a 1,000-page book and someone asks for a summary. You wouldn't give them all 1,000 pages; you'd pick the most important sentences and combine them. That is exactly what pooling does to a feature map.

- **No Kernels Here:** Unlike convolution, pooling doesn't use a kernel with learned weights. It just uses a simple math rule (like "pick the biggest number") over a sliding window.
- **Depth Stays the Same:** Pooling operates on every "sheet" or channel of your image independently. It shrinks the width and height but keeps the number of channels exactly the same.

The Three Common Types of Pooling

- **Max Pooling:** This is the most popular choice. As the window slides, the computer only keeps the maximum (largest) value. It captures the most "prominent" or strongest features. If a detective found a very sharp edge, Max Pooling makes sure that edge stays visible even as the image gets smaller. The resulting image is often sharper than the original.
- **Average Pooling:** This layer calculates the average value of all the numbers in the window. It keeps the essence of the area but smoothens everything out. It's like looking at a photo through a slightly blurry lens; you lose the sharpest details but keep the general colors and shapes.
- **Global Pooling:** Sometimes, instead of shrinking the image piece-by-piece, we use Global Average Pooling at the very end of the network. It takes an entire feature map and turns it into one single number by averaging every pixel in that map. It is often used to replace the "Flattening" step we saw earlier, helping the model stay simpler and more efficient.

Why do we need to pool?

If we didn't have pooling, our neural network would become a "memory hog."

Pooling helps us by:

- **Reducing Computation:** Fewer pixels mean the computer has to do less math in the next layer.
- **Adding Translational Invariance:** If a feature (like an eye) moves slightly to the left or right, a "Max Pool" will likely still grab that high value, helping the computer recognize the object even if it's shifted.
- **Improving Generalization:** It prevents the computer from becoming too obsessed with tiny, specific details (overfitting) and helps it focus on the bigger patterns.

POOLING LAYER

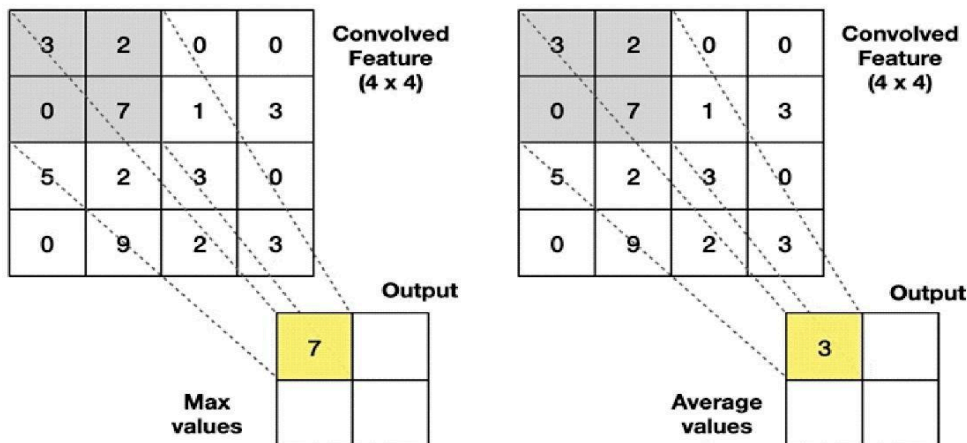
Max Pooling

Take the **highest** value from the area covered by the kernel

Average Pooling

Calculate the **average** value from the area covered by the kernel

Example: Kernel of size 2 x 2; stride=(2,2)



Source: <https://towardsdatascience.com/convolutional-neural-networks-explained-how-to-successfully-classify-images-in-python-df829d4ba761/>

FLATTENING

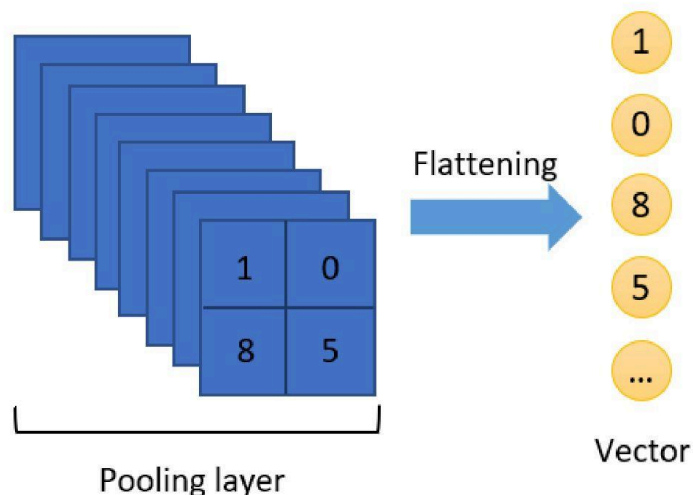
After convolutional and pooling layers have extracted relevant features from the input image, we have to turn this high-dimensional feature map into a format suitable for feeding into fully connected layers (which acts as the classifier/decision maker).

At this point, we have a grid of data (pixels in a feature map), and we want to line up all of these grid points in a single, long line. That's what **Flattening** does. It **takes the entire feature map** and **reorganizes it into a single, long vector**.

Why do we need flattening layers?

- **Integration of features:** By flattening the feature maps into a vector, the network can integrate the spatially distributed features extracted for tasks such as classification.
- **Compatibility with Dense Layers:** Fully connected layers (dense layers) are designed to operate on 1-dimensional data. Hence, flattening is a necessary step to transition from the multidimensional tensors produced by convolutional layers to the format required for dense layers.

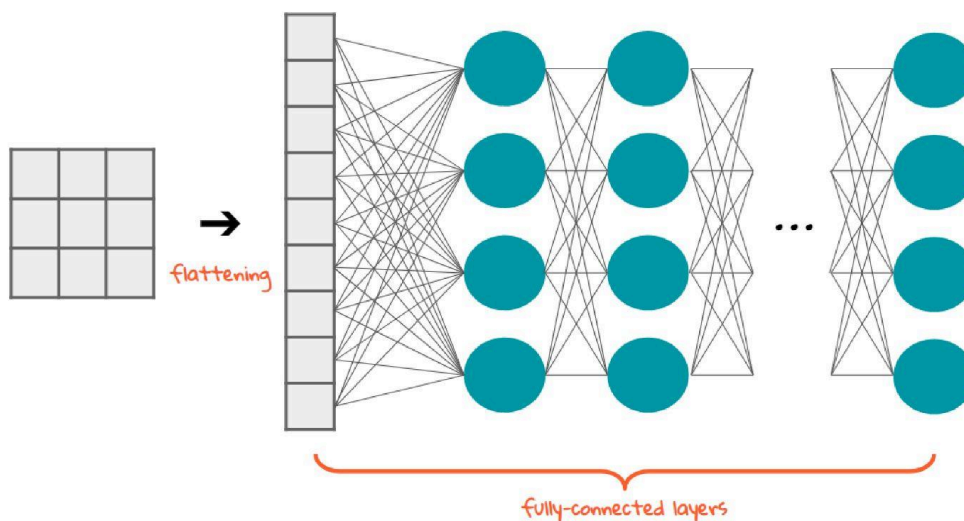
Although flattening changes the shape of the data, it does not make any changes to the actual information.



Source: <https://blog.gopenai.com/understanding-autoencoders-part-iv-the-fully-connected-layer-and-flattening-566f246fa1a>

FULLY CONNECTED LAYER / DENSE LAYER

Typically, **at least one** Dense Layer is needed at the end of a CNN architecture. However, the number of dense layers and their neurons can vary widely.



Source: <https://mohamedbakrey094.medium.com/a-simplified-explanation-of-cnn-component-layers-in-deep-learning-518b5c45ec8c>

4

Think of this like a courtroom. The detectives have brought in all the evidence (features like eyes, whiskers, and ears), and now the jury needs to sit down, look at everything together, and reach a final verdict: "Is this a cat or a dog?"

What are Fully Connected Layers/Dense Layers?

Dense Layers are the parts of the network that look exactly like the standard feedforward neural networks we discussed earlier.

- **The Bridge:** They take the high-level, abstract "clues" from the Flattening layer (which turns 2D maps into a long 1D list of numbers).
- **The Decision-Maker:** While the convolutional layers are great at "seeing" (detecting features), the Dense layers are essential for integrating those features into a final prediction.
- **Global Integration:** Because every neuron in a Dense layer connects to every single neuron from the previous layer, they can combine information from different corners of the image that the convolution layers might have processed separately.

The Purpose: Turning "Seeing" into "Knowing"

Why can't we just stop at the convolution layers? Because the convolution layer just says, "I see a circle here." The Dense layer is the one that says, "Because I see two circles at the top and a triangle in the middle, this must be a face".

- **Feature Combination:** They turn abstract features into concrete predictions.
- **Global Context:** They can integrate spatial information that pooling and convolution might have missed by looking at the "big picture" all at once.

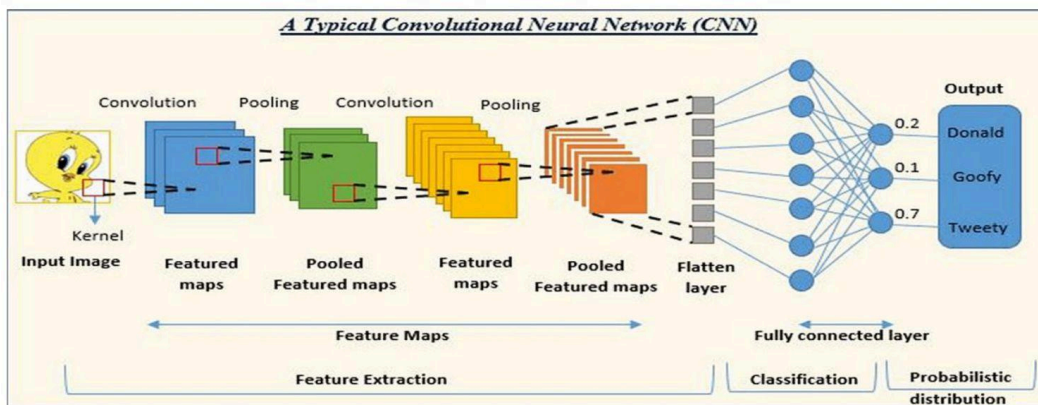
Output Layer

The very last Dense layer in your network is the Output Layer. Its job depends on what you want the computer to do:

- **Classification (Picking a Category):** If you are teaching the computer to recognize animals, the output layer uses an activation function like SoftMax or Sigmoid to give you a score (like "90% sure it's a dog").
- **Regression (Predicting a Number):** If you want the computer to predict something continuous, like the price of a house or the age of a person, the layer outputs a single continuous value.

A TYPICAL CNN STRUCTURE

We can put all these layers together to form a whole CNN structure. Typically, a CNN starts with some **Convolutional + Pooling layers** (Conv. layers followed by Pooling) → **Flattening** → Followed by a few **Dense layers** → **Output layer**. Activations are used after each Conv./Dense layer, and Batch Normalization and Dropout layers are often added in between to enhance performance and generalization.



Source: <https://python.plainenglish.io/understanding-cnns-and-resnets-a-beginners-guide-with-python-code-f4afb19cf4e4>

INPUT/OUTPUT DIMENSIONS

1. Convolutional Layer

Let, Input Dimensions $\rightarrow [W_{in}, H_{in}, C_{in}]$

Output Dimensions $\rightarrow [W_{out}, H_{out}, C_{out}]$

Then,

$$W_{out} = \frac{W_{in} - F + 2 * P}{\#Stride} + 1$$
$$H_{out} = \frac{H_{in} - F + 2 * P}{\#Stride} + 1$$
$$C_{out} = \text{Number of Filters applied on the input images}$$

Where, F = Kernel Dimension (F for an $F * F$ kernel)

P = Padding Length

$\#Stride$ = Strides (Default stride is 1 for Conv layers)

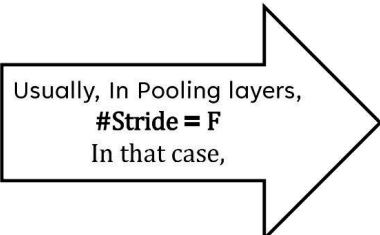
2. Pooling Layer

Let, Input Dimensions $\rightarrow [W_{in}, H_{in}, C_{in}]$

Output Dimensions $\rightarrow [W_{out}, H_{out}, C_{out}]$

Then,

$$W_{out} = \frac{W_{in} - F + 2 * P}{\#Stride} + 1$$
$$H_{out} = \frac{H_{in} - F + 2 * P}{\#Stride} + 1$$
$$C_{out} = C_{in}$$



Usually, In Pooling layers,
 $\#Stride = F$
In that case,

$$W_{out} = \frac{W_{in} + 2 * P}{F}$$
$$H_{out} = \frac{H_{in} + 2 * P}{F}$$
$$C_{out} = C_{in}$$

Where, F = Kernel Dimension (F for an $F * F$ Pooling kernel)

P = Padding Length

$\#Stride$ = Strides

3. Flattening

Let, Input Dimensions $\rightarrow [W_{in}, H_{in}, C_{in}]$

Then,

Output Dimensions $\rightarrow W_{in} \times H_{in} \times C_{in} \rightarrow \#output \text{ units}$

4. Fully Connected (Dense) Layer

Let, Input Dimensions $\rightarrow$ #input units (neurons)

Then,

Output Dimensions $\rightarrow$ #output units (neurons)

PARAMETER CALCULATION

1. Convolutional Layer

For a convolutional layer, the number of parameters is determined by the size of the kernels, the number of filters, and the number of input channels.

We can calculate the number of parameters in the following manner:

$$\text{\#Parameters} = \{(k_w \times k_h \times C_{in}) + 1\} \times C_{out}$$

Where:

- k_w = kernel width
- k_h = kernel height
- C_{in} = number of input channels
- C_{out} = number of filters (output channels)

The “+1” accounts for the bias term for each filter.

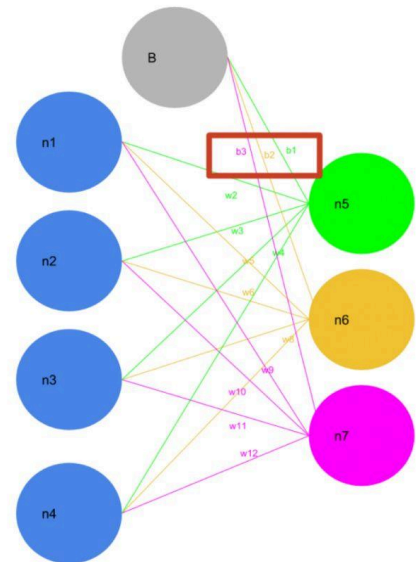
2. Fully Connected (Dense) Layers

For a fully connected layer, the number of parameters is given by the number of input units(neurons) times the number of output units(neurons), plus one bias term for each output unit(neuron).

$$\text{\#Parameters} = (\text{\#input units} + 1) \times \text{\#output units}$$

Let's take an example, where the fully connected layer has **128 input layer units** and **64 output layer units**. The computed parameters are:

$$\text{\#Parameters} = (128 + 1) \times 64 = 129 \times 64 = 8256$$



3. Pooling Layers

Pooling layers (e.g., max pooling, average pooling) do not have learnable parameters, so they contribute **0** to the parameter count.

4. Batch Normalization Layers

For batch normalization layers, each feature channel has two parameters (**gamma** and **beta**).

$$\#Parameters = 2 \times C_{out}$$

If a batch normalization layer is applied after a Fully Connected layer, then,

$$\#Parameters = 2 \times \#output\ units$$

Let's suppose that the batch normalization layer is applied to an output of **64** channels. Then the number of parameters can be computed as:

$$\#Parameters = 2 \times 64 = 128$$

N.B: In practical scenarios, there are actually **four** parameters (**gamma**, **beta**, and the **moving mean** and **variance**) for BN layers. However, the latter two are **non-trainable parameters**.

EXAMPLE 1

Layer	Input Dimensions	Filter Size	#Filters or, #Neurons	Padding	Output Dimensions	#Params
Conv1	128 * 128 * 3	5*5	32	2		
MaxPool1		2*2	—	0		
Conv2		3*3	64	0		
MaxPool2		2*2	—	0		
Flatten		—	—	—		
FC		—	128	—		
FC (Output)		—	4	—		

The above table represents a CNN architecture and its number of parameters in each layer. **Complete the table** and Comment on the **number of classes**.

Example1:

Layer	Input Dimension	Kernel Size	Filters or Neurons	Padding	Output	Parameter
Conv1	128*128*3	5*5	32	2	128*128*32	$((5*5*3)+1)*32 = 2432$

MaxPool1	128*128*32	2*2	–	0	64*64*32	0
Conv2	64*64*32	3*3	64	0	62*62*64	18496
MaxPool2	62*62*64	2*2	–	0	31*31*64	0
Flatten	31*31*64	–	–	–	61504	0
FC	61504	–	128	–	128	$(61504*128)+128 = 7872640$
FC(Output)	128	–	4	–	4	516

This is a multiclass classification problem. Softmax activation function is used in the output.

Example2: If Batch Size (e.g. 8) is mentioned in the question,

Layer	Input Dimension	Kernel Size	Filters or Neurons	Padding	Output	Parameter
Conv1	8*128*128*3	5*5	32	2	8*128*128*32	$((5*5*3)+1)*32 = 2432$
MaxPool1	8*128*128*3 2	2*2	–	0	8*64*64*32	0
Conv2	8*64*64*32	3*3	64	0	8*62*62*64	18496
MaxPool2	8*62*62*64	2*2	–	0	8*31*31*64	0
Flatten	8*31*31*64	–	–	–	8*61504	0
FC	8*61504	–	128	–	8*128	$(61504*128)+128 = 7872640$
FC(Output)	8*128	–	4	–	8*4	516

When we process a batch of 8 images, the model uses the exact same weights to analyze every single image in that batch. We don't create 8 sets of weights; we apply one set of weights to 8 different inputs simultaneously. So, we don't have to multiply the parameters with batch size.

CNN

Question 4

Alice is a BRACU student and she is taking CSE428 this semester. For her final project, she is trying to implement a CNN architecture for a classification task that comprises of the following layers:

Layer	Input Dimensions	Filter Size	#Filters or, #Neurons	Padding	Output Dimensions	#Params
Conv1	128 * 128 * 3	7*7	8	2		
MaxPool1		2*2	—	0		
Conv2		5*5	16	2		
MaxPool2		2*2	—	0		
Conv3		3*3	32	0		
AvgPool3		4*4	—	0		
Flatten		—	—	—		
FC		—	256	—		
FC		—	128	—		
FC (Output)		—	4	—		

In the table above, *Conv-X* denotes a Convolutional layer, *Pool-X* denotes a Pooling layer and *FC* denotes a Fully Connected layer.

- Determine the number of classes and the activation function used in the final layer.
- Calculate the input and output dimensions for each of the layers. (Complete the 2nd and 6th columns of the table).
- Calculate the number of Parameters for each of the layers. (Complete the last column of the table).
- Repeat Question b considering a Mini-Batch size of 32 instead of individual inputs.
- Suppose, Alice used Batch Normalization layers after each Convolutional and Fully Connected layer. Would it Change the total number of trainable parameters? If Yes, then, by how much?

CNNQuestion 4

Layer	Input Dimensions	Filter Size	# Filters or Neurons	Padding	Output Dimensions	Parameters
Conv1	$128 * 128 * 3$	$7 * 7$	8	2	$126 * 126 * 8$	$((7 * 7 * 3) + 1) * 8 = 1184$
MaxPool1	$126 * 126 * 8$	$2 * 2$	—	0	$63 * 63 * 8$	0
Conv2	$63 * 63 * 8$	$5 * 5$	16	2	$63 * 63 * 16$	$((5 * 5 * 8) + 1) * 16 = 3216$
MaxPool2	$63 * 63 * 16$	$2 * 2$	—	0	$31 * 31 * 16$ <i>↳ floor value</i>	0
Conv3	$31 * 31 * 16$	$3 * 3$	32	0	$29 * 29 * 32$	$((3 * 3 * 16) + 1) * 32 = 4640$
AvgPool3	$29 * 29 * 32$	$4 * 4$	—	0	$7 * 7 * 32$ <i>↳ $\lfloor 29/4 \rfloor$</i>	0
Flatten	$7 * 7 * 32$	—	—	—	$7 * 7 * 32 = 1568$	0
FC	1568	—	256	—	256	$(1568 * 256) + 256 = 401664$
FC	256	—	128	—	128	$(256 * 128) + 128 = 32896$
FC (Output)	128	—	4 <i>multiclass classification</i>	—	4	$(128 * 4) + 4 = 516$

Last activation $\rightarrow$ Softmax (multiclass classification)

In that case, Output dimension = Filters

Binary classification (filter = 2) $\rightarrow$ Sigmoid

In that case, Output dimension = 1

Convolutional Layer

$$W_{out} = \frac{W_{in} - F + 2 * P}{\# \text{ stride}} + 1$$

$$= \frac{128 - 7 + 2 * 2}{1} + 1$$

$$= 126$$

$$H_{out} = \frac{H_{in} - F + 2 * P}{\# \text{ stride}} + 1$$

$$= \frac{128 - 7 + 2 * 2}{1} + 1$$

$$= 126$$

C_{out} = Number of Filters applied on the input images.

$$= 8$$

Input Dimensions

→ $[W_{in}, H_{in}, C_{in}]$

For Batch Size $[N, W_{in}, H_{in}, C_{in}]$

Output Dimensions

→ $[W_{out}, H_{out}, C_{out}]$

For Batch size $[N, W_{out}, H_{out}, C_{out}]$

F = Kernel Dimension

(F for an $F * F$ kernel)

P = Padding Length

$\# \text{ Stride} = \text{strides}$

$$= 1$$

Pooling Layer

$$W_{out} = \frac{W_{in} + 2 * P}{F}$$

$$= \frac{126 + 2 * 0}{2} = 63$$

$$H_{out} = \frac{H_{in} + 2 * P}{F}$$

$$= \frac{126 + 2 * 0}{2} = 63$$

$$C_{out} = C_{in} = 8 \text{ (no change)}$$

Input Dimension →

$[W_{in}, H_{in}, C_{in}]$

Output Dimension →

$[W_{out}, H_{out}, C_{out}]$

$\# \text{ Stride} = F$

Flattening

Input Dimensions → $[W_{in}, H_{in}, C_{in}]$

Then, Output Dimensions → $W_{in} \times H_{in} \times C_{in} \rightarrow \# \text{ output units}$

Parameter Calculation

Convolutional Layer :

$$\text{Parameters} = \{(k_w \times k_h \times c_{in}) + 1\} \times C_{out}$$

where :

k_w = kernel width

k_h = kernel height

c_{in} = number of input channels

C_{out} = number of filters (output channels)

The "+1" accounts for the bias term for each filter.

Fully connected Layers :

$$\text{Parameters} = (\# \text{ input units} \times \# \text{ output units}) + \# \text{ output units}$$

$$= (128 \times 64) + 64 = 8256$$

→ 128 input layer units

→ 64 output layer units

Pooling Layer :

$$\text{Parameter} = 0$$

Batch Normalization layer :

$$\text{Parameters} = 2 \times C_{out}$$

If a batch normalization layer is applied after a fully connected layer,

$$\text{Parameters} = 2 \times \# \text{ output units}$$

a)

Number of classes: There are 4 classes, as indicated by the 4 neurons in the FC (output) layer.

Activation Function: For a multi-class classification task with more than 2 classes, the softmax activation function is used in the final layer.

d)

Layer	Input Dimensions	Output Dimensions
conv 1	$32 \times 128 \times 128 \times 3$	$32 \times 126 \times 126 \times 8$
Maxpool 1	$32 \times 126 \times 126 \times 8$	$32 \times 63 \times 63 \times 8$
conv 2	$32 \times 63 \times 63 \times 8$	$32 \times 63 \times 63 \times 16$
Maxpool 2	$32 \times 63 \times 63 \times 16$	$32 \times 31 \times 31 \times 16$
conv 3	$32 \times 31 \times 31 \times 16$	$32 \times 29 \times 29 \times 32$
Avgpool 3	$32 \times 29 \times 29 \times 32$	$32 \times 7 \times 7 \times 32$
Flatten	$32 \times 7 \times 7 \times 32$	$32 \times 7 \times 7 \times 32$
FC	1568×32	32×25
FC	32×256	32×128
FC (output)	32×128	32×4

e) Yes, this changes the number of trainable parameters.

A standard Batch Normalization layer has 4 parameters per channel/neuron: two trainable (γ and β) and two non-trainable (moving mean and variance).

Calculations for Trainable Parameters:

$$\text{Parameter} = 2 \times \text{Count} \\ \hookrightarrow \text{number of filters}$$

$$\text{After conv1} : 2 \times 8 = 16$$

$$\text{After conv2} : 2 \times 16 = 32$$

$$\text{After conv3} : 2 \times 32 = 64$$

$$\text{After FC (256)} : 2 \times 256 = 512$$

$$\text{After FC (128)} : 2 \times 128 = 256$$

$$\text{Total increase} : 16 + 32 + 64 + 512 + 256 = 880$$

additional trainable
parameters.

With Dropout Rate 0.5%

Layer	Input Dimension	Kernel Size	Filters or Neurons	Padding	Output	Parameter
Conv1	128*128*3	7*7	8	2	126*126*8	$((7*7*3)+1)*8 = 1184$
MaxPool1	126*126*8	2*2	—	0	63*63*8	0
Conv2	63*63*8	5*5	16	2	63*63*16	3216
MaxPool2	63*63*16	2*2	—	0	31*31*16	0
Conv3	31*31*16	3*3	32	0	29*29*32	4640
AvgPool3	29*29*32	4*4	—	0	7*7*32	0
Flatten	7*7*32	—	—	—	1568	0
FC	1568	—	256	—	256	401664
Dropout	256	—	0.5%	—	256	0
FC	256	—	128	—	128	32896
FC(Output)	128	—	4	—	4	516

Layer	Input Dimensions	Filter Size	#Filters or, #Neurons	Padding	Output Dimensions	#Params
Conv1	128 * 128 * 3	5*5	32	2		
MaxPool1		2*2	—	0		
Conv2		3*3	64	0		
MaxPool2		2*2	—	0		
Flatten		—	—	—		
FC		—	128	—		
FC (Output)		—	4	—		

Refer to the same table and **determine the dimension of the receptive field of a neuron in the Convolutional layer 2 (Conv2)**. What would be the dimensions if the Max Pooling layer 1 (MaxPool1) was removed?

1. Receptive Field of Conv2 (With MaxPool1)

We use the formula:

$$RF_{\text{current}} = RF_{\text{previous}} + (k - 1) * S_{\text{previous}}$$

Where k is the kernel size and S_{previous} is the cumulative stride of all layers before it.

- Input Layer: $RF = 1$, $Stride = 1$
- Conv1 (5 x 5, Stride 1): $RF = 1 + (5 - 1) * 1 = 5$
 - Cumulative Stride = $1 \times 1 = 1$
- MaxPool1 (2 x 2, Stride 2): $RF = 5 + (2 - 1) * 1 = 6$
 - Cumulative Stride = $1 \times 2 = 2$
- Conv2 (3 x 3, Stride 1): $RF = 6 + (3 - 1) * 2 = 10$

The receptive field dimension of a neuron in Conv2 is 10×10.

2. Receptive Field of Conv2 (If MaxPool1 was REMOVED)

If we remove the pooling layer, we will lose the stride that "multiplies" the reach of subsequent kernels.

- Input Layer: $RF = 1$, $Stride = 1$
- Conv1 (5 x 5, Stride 1): $RF = 1 + (5 - 1) * 1 = 5$
 - Cumulative Stride = $1 \times 1 = 1$
- Conv2 (3 x 3, Stride 1): $RF = 5 + (3 - 1) * 1 = 7$

If MaxPool1 is removed, the dimensions would be 7×7.

CNN ARCHITECTURE DESIGN

We have already discussed how a typical CNN architecture looks like. However, a convolutional neural network structure can vary widely (unlimited variations possible). So, it is important to structure a CNN in an intuitive and practical way. Different choices affect model capacity, overfitting, and performance.. So, a CNN must be designed considering all these factors.

General Structure of a CNN

A typical CNN has 3 main parts:

- **Input Layer:** Accepts image of shape: $(H \times W \times C)$ — e.g., $(224 \times 224 \times 3)$ for RGB).
- **Feature Extraction Block:** Repeating pattern consisting of convolutional blocks: **[Conv → Activation (ReLU) → (Optional BatchNorm) → Pooling]**. This block is stacked multiple times, often with increasing filter counts.
- **Classification Head:** **[Flatten or Global Average Pooling → Dense Layer(s) → Output]**. Often ends with SoftMax (classification) or sigmoid (binary).

The table below summarizes the most important design choices, trends and heuristics for convolutional neural networks.

Component	Key Considerations
# of Conv Layers	More layers = better abstraction but harder to train.
Filter Size	3×3 is common (captures local patterns); sometimes 1×1 or 5×5.
# of Filters	Increases with depth (e.g., 32 → 64 → 128) to capture complex features.
Padding	'Same' (output same size) vs. 'valid' (shrinks output).
Strides	Controls downsampling speed (use 2 in place of pooling sometimes).
Pooling	Max pooling for sharp features; Avg pooling for smooth ones.
Activation	ReLU is default; alternatives: Leaky ReLU, ELU.
Batch Normalization	Stabilizes training, allows higher learning rate.
Dropout	Reduces overfitting (especially in dense layers).

We have a sample 5-layer CNN architecture for classifying 224x224x3 images into 10 classes on the right. This architecture can be further improved by accommodating batch normalization layer after the convolutional layers.

Designing a CNN is like crafting a visual processing pipeline. Early layers capture details, deeper layers abstract, and the classifier makes the decision. Smart choices in **filter size**, **layer depth**, and **regularization** make all the difference.

Some Key Design Intuitions

- Early layers learn edges and textures, deeper layers learn object parts.
- Deeper networks are powerful but more prone to vanishing gradients and overfitting.
- Using global average pooling instead of flatten+dense can reduce overfitting.

***Think:** If you were designing a CNN to classify hand-written digits vs. cancer tumors in MRI scans, how would your architecture differ?*

```
graph TD
    Input[Input (224x224x3)] --> Conv1[Conv (3x3, 32 filters) → ReLU]
    Conv1 --> Pool1[Max Pooling (2x2)]
    Pool1 --> Conv2[Conv (3x3, 64 filters) → ReLU]
    Conv2 --> Pool2[Max Pooling (2x2)]
    Pool2 --> Conv3[Conv (3x3, 128 filters) → ReLU]
    Conv3 --> GAP[Global Average Pooling]
    GAP --> Dense1[Dense (128) → ReLU → Dropout]
    Dense1 --> Dense2[Dense (10) → SoftMax (Output)]
```